x	match the character 'x'
.	any character (byte) except newline
[xyz]	a "character class"; in this case, the pattern matches either an 'x', a 'y', or a 'z'
[abj-oZ]	a "character class" with a range in it; matches an 'a', a 'b', any letter from 'j' through 'o', or a 'Z'
[^A-Z]	a "negated character class", i.e., any character but those in the class. In this case, any character EXCEPT an uppercase letter.
[^A-Z\n]	any character EXCEPT an uppercase letter or a newline
r*	zero or more r's, where r is any regular expression
r+	one or more r's
r?	zero or one r's (that is, "an optional r")
r{2,5}	anywhere from two to five r's
r{2,}	two or more r's
r{4}	exactly 4 r's {name} the expansion of the "name" definition (see above)
"[xyz]\"foo"	the literal string: [xyz]"foo
\X	if X is an 'a', 'b', 'f', 'n', 'r', 't', or 'v', then the ANSI-C interpretation of \x. Otherwise, a literal 'X' (used to escape operators such as '*')
\0	a NUL character (ASCII code 0)
\123	the character with octal value 123
\x2a	the character with hexadecimal value 2a
(r)	match an r; parentheses are used to override precedence (see below)

rs	the regular expression r followed by the regular expression s; called "concatenation"
r|s	either an r or an s
r/s	an r but only if it is followed by an s. The text matched by s is included when determining whether this rule is the "longest match", but is then returned to the input before the action is executed. So the action only sees the text matched by r. This type of pattern is called trailing context".
^r	an r, but only at the beginning of a line (i.e., which just starting to scan, or right after a newline has been scanned).
r$	an r, but only at the end of a line (i.e., just before a newline). Equivalent to "r/\n".
<s>r	an r, but only in start condition s (see below for discussion of start conditions)
<s1,s2,s3>r	same, but in any of start conditions s1, s2, or s3 <*>r an r in any start condition, even an exclusive one.
<<EOF>>	an end-of-file
<s1,s2><<EOF>>	an end-of-file when in start condition s1 or s2